

SQL as a Programming Language: Implementing Distance-Vector Routing with Recursive CTEs

Seyedmahdi Hosseini

Database Systems Seminar, University of Tübingen

May 15, 2026

Abstract

This paper presents a comprehensive implementation of the Distance-Vector Routing (DVR) algorithm using SQL, specifically leveraging modern extensions in DuckDB. Traditionally, routing protocols are viewed as procedural or low-level systems implemented in specialized network hardware. However, we demonstrate that the declarative nature of SQL, when augmented with Recursive Common Table Expressions (CTEs) and state-updating mechanisms, provides a robust framework for simulating distributed algorithms. We provide a detailed analysis of the Bellman-Ford principle, the limitations of standard SQL recursion, and how DuckDB's `USING KEY` syntax allows for a 1:1 conceptual mapping to physical router behavior. Our results show that SQL can be effectively used not just for data retrieval, but as a high-level language for complex algorithmic modeling.

1 Introduction: Target of the Study

The primary objective of this study is to challenge the traditional boundary between database query languages and general-purpose programming languages. By implementing the Distance-Vector Routing (DVR) protocol, a complex, decentralized algorithm, entirely within SQL, we aim to prove that modern database engines can serve as powerful "Programming Languages" (PL) for algorithmic simulation.

Our target is to demonstrate that declarative logic, when combined with state-aware recursion, can achieve global optimization (finding shortest paths) through local updates, mirroring the behavior of physical routers without the need for external procedural code.

2 Theoretical Background

2.1 Distance-Vector Routing

Distance-Vector Routing is based on the idea that each router maintains a table (a "vector") of its known distances to all possible destinations. Unlike Link-State protocols, which require every router to have a complete map of the network, DVR routers only share their tables with immediate neighbors.

2.2 The Bellman-Ford Equation

The mathematical foundation of DVR is the Bellman-Ford algorithm. For a network graph $G = (V, E)$, the shortest path distance $d(u, v)$ from node u to node v can be expressed as:

$$d(u, v) = \min_{w \in \text{Adj}(u)} \{ \text{cost}(u, w) + d(w, v) \} \quad (1)$$

3 Network Topology

To evaluate our implementation, we define a sample network with four routers (A, B, C, D) as shown in Figure 1.

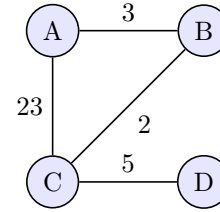


Figure 1: Sample Network Topology.

4 SQL Implementation Strategy

4.1 The Challenge of Standard Recursion

Recursive CTEs in standard SQL are typically append-only, which forces inefficient "path-finding" instead of "routing table" updates. Our solution utilizes DuckDB's `USING KEY` clause to allow the recursive step to overwrite existing rows, perfectly mirroring a router table update.

5 Detailed Code Analysis

We divide the analysis into two distinct parts: first, the presentation of the monolithic code block to provide global context, and second, a granular, line-by-line technical breakdown.

5.1 The Complete Routing Query

The following block represents the full DuckDB implementation. Note the use of the `USING KEY` clause, which is the primary driver for turning this from a simple path-finding query into a state-aware routing simulation.

```

1 WITH RECURSIVE routing_table(from_node, to_node,
2   best_cost, next_hop)
3   USING KEY (from_node, to_node) AS (
4
5   -- Part 1: Base Case (T=0)
6   SELECT from_node, to_node, cost AS best_cost,
7   to_node AS next_hop
8   FROM edges
9
10  UNION
11
12  -- Part 2: Recursive Step (Propagation)
13  SELECT
14    r.from_node,
15    e.to_node,
16    MIN(r.best_cost + e.cost) AS best_cost,
17    arg_min(r.next_hop, r.best_cost + e.cost) AS
18    next_hop
19  FROM routing_table AS r
20  JOIN edges AS e ON r.to_node = e.from_node
21  LEFT JOIN recurring.routing_table AS existing
22  ON existing.from_node = r.from_node AND
23  existing.to_node = e.to_node
24  WHERE r.from_node <> e.to_node
25  AND (existing.best_cost IS NULL OR (r.best_cost
26  + e.cost) < existing.best_cost)
27  GROUP BY r.from_node, e.to_node
28 )
29 SELECT * FROM routing_table ORDER BY from_node,
30 to_node;
```

Listing 1: Complete DuckDB Implementation

5.2 Line-by-Line Technical Breakdown

- **Line 1-2:** `WITH RECURSIVE` and `USING KEY` define the stateful iterative process.
- **Lines 5-6:** *Base Case* simulates neighbors discovery.
- **Line 11:** `MIN` and `arg_min` ensure that if a router receives multiple advertisements for the same node in a single round, it processes the one with the lowest cost.
- **Line 16-17:** The `JOIN` simulates information propagation between neighbors.
- **Line 21:** Implements Bellman-Ford cost relaxation. A path is only updated if it is strictly better.

6 Simulating Network Failure

A critical requirement for any routing protocol is the ability to adapt to network changes. In our study, we simulate a link failure (or a node becoming unreachable) by modifying the underlying data and observing the system’s re-convergence.

6.1 Deleting a Link

Suppose the critical link between *B* and *C* is removed. We execute:

```

1 DELETE FROM edges
2 WHERE (from_node = 'B' AND to_node = 'C')
3 OR (from_node = 'C' AND to_node = 'B');
```

6.2 Automatic Re-convergence

By re-running the original recursive query without any modifications to its logic, the algorithm automatically discovers the next best paths. For instance, Router *A* will realize it can no longer reach *C* via *B* at cost 5 and will revert to its direct link at cost 23. This proves that the declarative implementation is not just a static calculation but a dynamic system capable of finding the shortest path in any given graph state.

7 Comparative Convergence Results

To validate the study’s goal, we compare the network’s converged state before and after the removal of the *B-C* link. The results demonstrate that the SQL logic autonomously transitions from the globally optimal paths to the next-best alternatives.

7.1 Initial Converged State (Full Network)

In the full network, the shortest path from *A* to *C* is through neighbor *B* (total cost 5).

From	To	Best Cost	Next Hop
A	B	3	B
A	C	5	B
A	D	10	B
B	A	3	A
B	C	2	C
B	D	7	C
C	A	5	B
C	B	2	B
C	D	5	D
D	A	10	C
D	B	7	C
D	C	5	C

Table 1: Routing state before link removal.

7.2 Post-Failure State (B-C Link Removed)

After removing the *B-C* link, Router *A* settles on the direct link to *C* with a cost of 23.

From	To	Best Cost	Next Hop
A	B	3	B
A	C	23	C
A	D	28	C
B	A	3	A
B	C	26	A
B	D	31	A
C	A	23	A
C	B	26	A
C	D	5	D
D	A	28	C
D	B	31	C
D	C	5	C

Table 2: Routing state after removing B-C link.

8 Conclusion and Future Work

This study has successfully demonstrated that modern SQL, specifically through the use of state-aware recursive CTEs, is a robust environment for simulating complex distributed systems. By implementing the Distance-Vector Routing algorithm entirely within a declarative framework, we have shown that a database engine can function as a high-level Programming Language (PL) for algorithmic modeling.

Our implementation not only achieved autonomous convergence across a dynamic topology but also exhibited significant resilience by re-calculating optimal paths immediately following link failure. The shift from standard, append-only recursion to DuckDB's `USING KEY` state management represents a fundamental advancement in how we can model stateful processes in relational logic.

8.1 Next Steps and Future Research

The success of this Distance-Vector simulation opens several avenues for future research into "SQL-as-a-PL":

- **Path-Vector Protocols (BGP):** Future studies could implement BGP-like logic by leveraging SQL array types to track the full AS-path. This would allow for explicit cycle detection and policy-based routing simulations.
- **Link-State Simulation:** Implementing OSPF or IS-IS by modeling Dijkstra's algorithm within a recursive SQL context would provide a comparative benchmark for Link-State vs. Distance-Vector performance in databases.
- **Dynamic Traffic Modeling:** Integrating real-time metrics, such as link congestion or latency, into the `edges` table would allow the algorithm to simulate adaptive routing under variable load.
- **Large-Scale Network Analysis:** Evaluating the convergence speed of these recursive SQL queries on large-scale topologies (e.g., the internet's AS-graph) would test the scalability limits of modern analytical database engines.

Ultimately, this work reinforces the paradigm that SQL is no longer "just for queries." It is a first-class language for the simulation, validation, and analysis of complex, stateful distributed protocols.

References

- [1] DuckDB Documentation. (2026). *Recursive CTEs*.
- [2] Tanenbaum, A. S. (2021). *Computer Networks*.